

Scilab

1 Wiadomości wstępne

Wykonaj poniższe przykłady, aby zapisać swoją pracę wywołaj polecenie `diary on` i `diary imię_nazwisko`. Na koniec zajęć wydaj polecenie `save nazwa_pliku`. Polecenie `diary off` wyłącza na czas do następnego wywołania `diary on`, zapisywanie do pliku.

Proste przykłady obliczeń w programie Scilab:

```
2+2
```

```
3*5
```

```
1/(2*3)
```

```
2^8
```

```
sqrt(4)
```

```
sin(10)
```

```
max(1,5,3,7,2)
```

Do powracania do poprzednich komend w konsoli służą strzałki w górę i w dół.

Wybór przykładowego katalogu, w którym znajdują się pliki Scilaba:

```
chdir('D:\Mój katalog\Scilab')
```

Katalog roboczy można również zmienić w oknie “Przeglądarka plików”.

Przejdźcie do nowej linii:

```
1+2+...
```

```
3+4
```

Gdy nie chcemy, aby wynik danej komendy był wyświetlany, stosujemy średnik:

```
a=1+2+3;
```

```
b=4+5+6;
```

```
a+b
```

Zmienne: Zmienne w Scilabie traktowane są jak macierze! Skalary są interpretowane jako macierze o wymiarze 1×1 .

`who`

- wyświetla listę zmiennych

`whos()`

- wyświetla listę zmiennych, ich wymiar, ile zajmują miejsca w pamięci. Wyświetla też listę nazw funkcji zdefiniowanych przez użytkownika

`clear a`

- usuwa zmienną `a`

`clear`

- usuwa wszystkie zdefiniowane przez użytkownika zmienne
Zmienne predefiniowane

`%pi`

`%eps`

`%i`

`%e`

`%inf`

`%nan`

`%T`

- wartość logiczna.

Definiowanie wektora:

`a=[1,2,3]`

Transponowanie wektora: (z zamianą na wartości sprzężone)

`a'`

lub "zwykłe" transponowanie:

`a.' //dawniej .a'`

Przykład dla macierzy:

`a=[1+%i,2+2*%i;3+3*%i,4+4*%i]`

`a'`

`a.'`

Iloczyn skalarny dwóch wektorów: Wykorzystujemy fakt, że

$$([1, 2, 3], [3, 2, 1]) = [1, 2, 3] \begin{bmatrix} 3 \\ 2 \\ 1 \end{bmatrix},$$

i wynik powyższego działania jest macierzą o wymiarze 1×1 , w myśl zasady, że iloczyn macierzy o k wierszach i m kolumnach i macierzy o m wierszach i n kolumnach jest macierzą o k wierszach i n kolumnach.

```
a=[1,2,3];  
b=[3,2,1];  
c=a*b' //10
```

W prosty sposób możemy zbudować tabliczkę mnożenia, np. do 10.

```
a=[1,2,3,4,5,6,7,8,9,10]  
a'*a
```

Specjalny operator umożliwiający mnożenie macierzy w sposób analogiczny do dodawania, tzn. element przez odpowiadający mu element:

```
a=[1,2,3]  
b=[3,2,1]  
c=a.*b
```

Analogicznie można “podzielić” dwa wektory:

```
a=[1,2,3]  
b=[3,2,1]  
c=a./b
```

Wektor o elementach równomiernie rozmieszczonych:

```
v = linspace(0,1,11)  
lub  
v = 0:0.1:1
```

Wprowadzanie macierzy (wielowierszowej):

```
a=[1,2;3,4]
```

Odwracanie macierzy:

```
inv(a)  
a^(-1)
```

- dzielenie lewo- i prawostronne
Potęgowanie:

```
^ lub **
```

“Specjalne” potęgowanie (każdego elementu osobno):

`.^`

Dodawanie liczby do macierzy powoduje dodanie jej do każdego elementu macierzy

Reszta z dzielenia:

`modulo(x,d)`

Przykład

`modulo(21,4)`

Operatory relacji

`~=`

`==`

`<`

`>`

`<=`

`>=`

Sprawdzimy ich działanie wpisując polecenia:

`A=rand(3,4)`

`B=A<0.5`

Wygenerowaliśmy losową macierz 3×4 i sprawdziliśmy, które jej elementy są mniejsze od 0,5.

Odwoływanie się do elementów macierzy, np:

`A(1,2)`

- element z pierwszego wiersza i drugiej kolumny

Odwoływanie się do fragmentów macierzy:

`A(1:2,1:2)`

lub

`A([1 2],[1 2])`

-wybieramy pierwsze dwa wiersze i pierwsze dwie kolumny.

i -ta kolumna macierzy, i -ty wiersz macierzy:

`A(:,i)`

`A(i,:)`

Suma elementów macierzy:

`sum(A)`

`sum(A,"r")`

`sum(A,"c")`

- dwie ostatnie sumy odpowiednio w wierszach i kolumnach
Skumulowane suma i iloczyn elementów macierzy:

```
cumsum(A)
cumprod(A)
```

Minimum i maksimum:

```
min(A)
max(A)
```

Średnia, odchylenie standardowe, wartości własne i wektory własne macierzy:

```
mean(A)
st_deviation(A)
spec(A)
bdiag(A)
```

Macierze specjalne:

```
eye(4,4)
diag([1,2,3])
zeros(4,5)
ones(2,3)
```

-macierz jednostkowa 4×4 , macierz z diagonalą 1, 2, 3 zawierająca same zera i zawierająca same jedynki.

Diagonala macierzy:

```
D=diag(M)
```

Macierze dolne i górnio trójkątne:

```
A = tril(B)
A = triu(B)
```

Wielomiany:

Zdefiniowanie wielomianu zmiennej z o współczynnikach 1, 2, 3.

```
w=poly([1, 2, 3],"z","coeff")
w =
```

$$1 + 2z + 3z^2$$

Dodawanie, odejmowanie, mnożenie, dzielenie wielomianów:

```
a+b a-b a*b a/b
```

Pochodna wielomianu

```
derivat(p)
```

Wartość wielomianu w punkcie x :

`horner(p,x)`

Pierwiastki wielomianu:

`roots(w)`

Wybrane funkcje:

`abs(x)`
`exp(x)`
`log(x)`
`log10(x)`
`sin(x)`
`cos(x)`
`tan(x)`
`cotg(x)`
`sqrt(x)`
`floor(x)`
`ceil(x)`

Wywołanie pomocy:

`help funkcja`

2 Programowanie

2.1 Podstawy

Instrukcja pusta

;

- Średnik lub pusta linia w skrypcie
Instrukcja przypisania

=

Sekwencja kilku instrukcji

;

- Średniki lub przecinki

2.2 Instrukcje warunkowe

`if wyrażenie_logiczne, instrukcja, end`

Przykład

```
a=5
b=4
if a>b, a=a+5, end
```

Skrypty: możemy je pisać w dowolnym edytorze lub w edytorze SciNotes (Narzędzia-SciNotes). W edytorze zapiszmy następujący tekst

```
if a>b then
  a=a+b
else
  a=a-b
end
```

Zapiszmy ten plik jako np. `iw.sce` (domyślne rozszerzenie dla skryptów) w naszym katalogu roboczym. Następnie wpisujemy w konsoli

```
exec iw.sce
```

Możemy też w edytorze SciNotes kliknąć przycisk Wykonaj albo Zapisz i wykonaj.

Wykonywane są instrukcje zapisane w pliku.
Zagęszczanie instrukcji warunkowej

```
if wyrażenie_logiczne_1 then
  ciąg_instrukcji_1
else
  if wyrażenie_logiczne_2 then
    ciąg_instrukcji_2
  else
    if wyrażenie_logiczne_3 then
      ciąg_instrukcji_3
    else
      ciąg_instrukcji_4
    end
  end
end
```

lub

```
if wyrażenie_logiczne_1 then
  ciąg_instrukcji_1
elseif wyrażenie_logiczne_2 then
  ciąg_instrukcji_2
elseif wyrażenie_logiczne_3 then
```

```

        ciąg_instrukcji_3
    else
        ciąg_instrukcji_4
    end
end

```

Przykład: badanie, czy wylosowana macierz jest rzeczywista czy zespolona

```

A=sqrt(-1*rand(3,3));
if imag(A)==0 then
    disp('A - macierz rzeczywista')
else
    disp('A - macierz zespolona')
end

```

2.3 Instrukcja wyboru

```

select identyfikator
case iden1
    ciąg_instrukcji_1
case iden2
    ciąg_instrukcji_2
...
else
    inny_ciąg_instrukcji
end

```

Zmienna identyfikator może być wartością liczbową lub łańcuchem.

Przykład:

```

n=round(10*rand(1,1));
select n
case 1
    disp(1);
case 2
    disp(2);
case 3
    disp(3);
else
    disp('inna wartosc');
end

```

2.4 Pętla for

```

for index=pocz:krok:koniec do
    ciąg_instrukcji;
end

```

(ewentualnie index=pocz:koniec z krokiem 1)

pocz - wartość początkowa zmiennej index
krok - przyrost zmiennej index (jeżeli jest równa 1, to można ją pominąć)
koniec - wartość końcowa zmiennej index
break - oznacza bezwarunkowe opuszczenie pętli
Przykład: 20!

```
a=1;
for n=1:20 do
    a=a*n;
end
disp(a)
```

Przykład 2:

$$\sum_{i=1}^{10} \frac{1}{i}$$

```
suma=0;
for i=1:10 do
    suma=suma+1/i;
end
disp(suma)
```

Przykład 3: wartość końcowa nie musi być osiągnięta

```
for k=1:4:10 do
    disp(k);
end
```

2.5 Pętla while

```
while wyrażenie_logiczne do
    ciąg_instrukcji
end
```

- pętla ta może być przerywana poleceniem break.

Przykład 1: suma liczb od 1 do 100:

```
a=0;
n=1;
while n<=100 do
    a=a+n;
    n=n+1;
end
disp(a)
```

3 Funkcje, skrypty

Uzupełnienie: komendy

```
savehistory("moj_plik.txt")
loadhistory("moj_plik.txt")
```

pozwalają na zachowywanie w plikach historii poleceń i przenoszenie jej np. na inny komputer.

3.1 Funkcje definiowane bezpośrednio

Jest to sposób definiowania funkcji ograniczony do prostych wyrażeń matematycznych i polega na bezpośrednim włączeniu kodu funkcji w program interpretowany lub umieszczeniu go w skrypcie. Do definiowania takiej funkcji służy polecenie `deff()`. Składnia:

```
deff('nazwa','wzor')
```

Przykład: $z = \sin x \cos x$

```
deff('y=z(x)','y=cos(x)*sin(x)');
z(10)
```

3.2 Funkcje zewnętrzne

Funkcje powinno się zapisywać w pliku z rozszerzeniem `*.sci`, ewentualnie `*.sce`. Zaleca się, aby nazwa pliku odpowiadała nazwie funkcji.

Struktura:

```
function[out1,out2,...]=nazwa_funkcji(in1,in2,...)
    ciąg_instrukcji
endfunction
```

Zmienne `out1`, `out2`, ... służą do odbierania obliczonych wewnątrz funkcji wielkości (mogą to być liczby, macierze i łańcuchy). Zmienne `in1`, `in2`, ... służą do wprowadzania danych do funkcji.

Przykład: funkcja licząca iloczyn liczb naturalnych od 1 do n .

```
function[il]=iloczyn(n)
    il=1
    for x=1:n do
        il=il*x
    end
endfunction
```

- zapisujemy ten tekst w pliku `iloczyn.sci`. Następnie w programie wpisujemy:

```
exec('iloczyn.sci');
iloczyn(5);
```

Zmienne globalne:

```
global a
```

Przykład: Obliczenie pola i obwodu koła o promieniu r

```
function[Po,Ob]=kolo(promien)
Po=%pi*promien^2;
Ob=2*%pi*promien;
endfunction
```

```
// wywołanie:
```

```
exec('kolo.sci');
[po ob]=kolo(6)
```

Funkcje rekurencyjne na przykładzie silni:

```
function[y]=silnia(n)
if n==0
    y=1
else
    y=n*silnia(n-1)
end
endfunction
```

Wykorzystanie polecenia return: (wyjście)

```
function[z]=dzielenie(x)
if x==0
    z=%inf
    return
end
z=1/x
endfunction
```

4 Przykłady wykorzystania funkcji bibliotecznych

4.1 Miejsca zerowe funkcji

Polecenie `fsolve(xstart,funkcja)` - znajduje rozwiązanie funkcji w otoczeniu punktu startowego $xstart$. Definiujemy funkcję $y = 2x^3 - 30x^2 - 3x + 200$.

```
function [y]=moja_funkcja(x)
y=2*x^3-30*x^2-3*x+200
endfunction
```

Rysujemy przebieg zmienności. Funkcja ma pierwiastki w okolicy punktów np. -1, 1, 11. Wywołujemy

```
x1=fsolve(-1,moja_funkcja)
x2=...
x3=...

function [y]=funkcja(a1,a2,a3,a4,a5,a6,a7)
    [lsh rhs]=argn()
    disp(rhs);
    y=123;
endfunction
```

Zauważmy, że funkcję można wywołać z różną ilością parametrów. Zmienna rhs oznacza liczbę przekazywanych do funkcji aktualnych parametrów wejściowych.

5 Wykresy

`plot(x,y)` gdzie x i y są wektorami, wynikiem jest wykres punktów o współrzędnych (x,y) branych kolejno z obu wektorów. Obydwa wektory muszą mieć jednakowe długości (ale nie jest istotne czy są wierszowe czy kolumnowe). Jeżeli x jest wektorem kolumnowym, a y jest macierzą (o tej samej co x liczbie wierszy), to otrzymamy tyle różnych wykresów ile macierz y ma kolumn i wszystkie one będą wykreślone względem wartości x . Jeśli x i y są macierzami (muszą mieć jednakowe rozmiary), to otrzymujemy tyle wykresów, ile macierze x i y mają kolumn.

Przykład 1:

```
x=[-5,-4,-3,-2,-1,0,1,2,3,4,5];
plot(x,x)
plot(x,x^2)
plot(x,x^3)
```

`clf()` - czyści okno graficzne `xdel()` - usuwa okno graficzne

Przykład 2:

```
t=[-%pi:0.01:%pi];
plot(t,sin(t))
```

- utworzenie wektora zawierającego dyskretne wartości zmiennej t i narysowanie wykresu funkcji $\sin t$.

Przykład 3:

```
x=linspace(0,2*%pi,101);
y=exp(-x.*sin(4*x));
plot(x,y);
```

- utworzenie wektora zawierającego 101 dyskretnych wartości zmiennej x i narysowanie wykresu funkcji $y(x) = e^{-x} \sin(4x)$.

Przykład 4:

```
x=[0.1:1:3*pi]';
y1=sin(x)./x;
y2=cos(x)./x;
plot(x, [y1 y2]);
```

- uwaga na transpozycję wektora w pierwszym poleceniu! x jest wektorem kolumnowym, $[y1 \ y2]$ jest macierzą o dwu kolumnach: $y1$ i $y2$. Otrzymujemy dwa wykresy.

Przykład 5:

```
x=[0:0.01:3*pi]';
plot2d(x,[sin(x) cos(x) x^2], style=[3,5,7])
```

Wykresy funkcji $\sin(x)$, $\cos(x)$ i x^2 . `plot2d` działa podobnie jak `plot`, ale jest wzbogacona o kilka elementów. Parametr `style` jest wektorem, którego wymiar jest równy ilości wykresów, a kolejne współrzędne oznaczają ich kolory. Oznaczenia kolorów: 1 - czarny, 2 - niebieski, 3 - zielony, 4 - jasnoniebieski, 5 - czerwony, 6 - różowy, 7 - czerwony, 8 - biały, 9 - granatowy, ... Ponadto parametr `style` z ujemnymi wartościami licząc od zera służy do ustawienia rodzaju linii: kropka, plusik, kreski itp.

Przykład 6:

```
x=linspace(-pi,pi,101);
plot2d(x,sin(x),rect=[-4,-1,4,2])
```

- `rect=[xmin,ymin,xmax,ymax]` - podaje zasięg osi x i y .

Przykład 7:

```
plot2d(x,sin(x),axesflag=4)
```

- `axesflag` określa sposób położenia osi i ramki wykresu, przyjmuje wartości od 0 do 5 i 9.

Przykład 8:

```
x=[-pi:0.01:pi]';
plot2d(x,[sin(x) cos(x) sin(x)^2],style=[3,5,7],...
leg="sinus@cosinus@kwadrat sinusa")
```

- opis wykresów (parametr `leg`). Dwie kropki - przeniesienie do następnej linii.

Przykład 9:

```
deff("[y]=f(x)","y=sin(x)+cos(x)")
x=[0:0.1:10]*pi/10;
fplot2d(x,f)
```

- rysowanie funkcji zdefiniowanej przez użytkownika - `fplot2d()`.

Przykład 10:

```

deff('z=f(x,y)', 'z=x^4-y^4')
x=-3:0.2:3;
y=x;
xtitle('Wykres własnej funkcji')
fplot3d(x,y,f,alpha=5,theta=31)

```

- rysowanie funkcji dwóch zmiennych w przestrzeni trójwymiarowej. xtitle - służy do ustawienia tytułu wykresu.

Przykład 11:

```

x=[-%pi:0.01:%pi]';
subplot(1,3,1);
plot2d(x,sin(x),style=3,leg="sinus")
subplot(1,3,2);
plot2d(x,cos(x),style=5,leg="cosinus")
subplot(1,3,3);
plot2d(x,sin(x)^2,style=7,leg="kwadrat sinusa")

```

- subplot(m,n,p) pozwala podzielić wykres na m wierszy i n kolumn. p określa, w którym miejscu okna będzie rysowany kolejny wykres (licząc wierszami).

Przykład 12: Ceny 20 losowo wybranych mieszkań w Lublinie kształtują się następująco (w tys. zł): 150, 198, 250, 499, 187, 345, 267, 189, 177, 176, 246, 389, 470, 267, 678, 654, 376, 278, 189, 768. Stworzymy histogram liczebności dla tych danych.

```

figure(1);
C = [150, 198, 250, 499, 187, 345, 267, 189, 177, 176,...
     246, 389, 470, 267, 678, 654, 376, 278, 189, 768];
histplot(10,C);
histplot(20,C);

```

- pierwszy parametr jest liczbą klas, drugi danymi. figure(nr) - pozwala na rysowanie w oknie o danym numerze (lub tworzy nowe).

Skrypt powstał po drobnych modyfikacjach ze skryptu dr Pawła Karczmarka

Literatura

- [1] A. Brozi, Podstawowe wiadomości o programie Scilab, skrypt internetowy
- [2] A. Brozi, Podstawowe wiadomości o programie Scilab - wykresy, skrypt internetowy
- [3] A. Brozi, Scilab w przykładach, NAKOM, Poznań 2007
- [4] H. Galda, Image Processing with Scilab and Image Processing Design Toolbox, 2011 - skrypt internetowy

- [5] C. T. Lachowicz, Matlab. Scilab. Maxima. Opis i przykłady zastosowań, Wydawnictwo Politechniki Opolskiej, Opole 2005
- [6] A. Paradzińska, Technologia informacyjna - materiały do zajęć